

Lab 3: Architectural Pattern Selection - Written Justification

Internal Microservice Catalog & Health Portal

We chose Microservices Architecture for the Internal Microservice Catalog & Health Portal System.

Architectural Choice

The portal is built as independently deployable services behind a single API Gateway: Catalog, Health Monitor, Dependency Graph, API Docs, Notification and Auth. Each data-owning service has its own data store (Catalog DB, Metrics DB, Graph DB, Docs Store), and services communicate only through the interfaces shown in the component diagram. We rejected Layered because health polling and the UI would share one deployable and scale and fail together. We rejected Client-Server because a single central server would be a single point of failure for a tool whose purpose is monitoring.

Two Reasons

- **1. Different load and failure profiles.** Health Monitor polls every registered service continuously, Catalog is read-heavy, and API Docs and Dependency Graph are used occasionally. With microservices each one scales and deploys on its own. If the API Docs Service crashes, health monitoring and service discovery keep working, which matters because a monitoring portal must stay available exactly when other systems are failing.
- **2. The portal's features are naturally separate domains.** Discovery, health monitoring, API documentation and dependency visualization each have their own data and rules. Mapping each to its own service lets us change or add a feature, for example adding the Notification Service for alerts, without redeploying or risking the rest of the system.

Security Advantage

The API Gateway is the only entry point. Every request is checked against the Auth Service (IAuth) before it is routed, and the internal services and their databases are never exposed directly. Each database is reachable only by the service that owns it, so a compromise of one service does not give access to the data of the others, which limits the blast radius and supports least-privilege access to internal service information.

Performance Benefit

The Health Monitor Service can be scaled horizontally on its own, and its polling runs independently of user-facing requests. Heavy health checks therefore do not slow down catalog browsing in the UI. Each service can also use the store that suits it best, such as a time-series store for health metrics and a graph store for dependencies, which keeps queries fast.

Trade-off Accepted

Microservices add network latency between services and more operational complexity (deployment, monitoring, data consistency). For this portal the gains in isolation, independent scaling and security outweigh these costs.